

Homework 2

Important Note: Do not distribute or share these problems. They are strictly for use by students registered in the Spring 2025 edition of CIS 3200 at University of Pennsylvania. Any violations will be referred to the Office of Student Conduct.

Note: The homework is due on **Monday, Feb 17 by 3:30 pm via Gradescope**. If you are using one of the two allowed extensions, then it is due on Wednesday, Feb 19 by 3:30 pm.

Please write concise and clear solutions; you will get only a partial credit for correct solutions that are either unnecessarily long or not clear. Whenever you present an algorithm, first give a precise description of your algorithm in English and only then present it in pseudocode if you think it is helpful. For most problems, pseudocode will be unnecessary. You should always argue the correctness of your algorithm and give an analysis of its complexity. Do remember to *typeset your homework*.

You are allowed to discuss ideas for solving homework problems in groups of up to 3 people but *you must write your solutions independently*. Also, you must write on your homework the names of the students with whom you discussed.

Finally, you are not allowed to use *any* material outside of the class notes and the textbook. Any violation of this policy may seriously affect your grade in the class.

Problem 1. Given a set S of $n \geq 2$ distinct positive integers, a pair $x, y \in S$ of distinct integers is called *similar* if $|x - y| \leq (A - B)/(n - 1)$, where A is the largest integer in S and B is the smallest integer in S . Design an $O(n)$ time algorithm to find a similar pair of integers in a given set S . (20 points)

Problem 2. You are given two strings A and B of length n , each consisting of possibly repeated characters from an alphabet of size M . A and B are said to be *anagrams* of one another if it is possible to reorder the characters in A to obtain string B (without removing or adding any characters). For instance, the strings “ELEVEN PLUS TWO” and “TWELVE PLUS ONE” are anagrams of one another. Your goal is to design an algorithm that decides if a given pair of strings are anagrams of one another. You may assume that all characters in the alphabet can be represented by distinct integers in the

range $[0..M)$, and that you can convert any character to its integer representation in unit time (note that M can be much larger than n).

Use universal hashing to design a randomized algorithm that takes $O(n)$ *worst-case time*, $O(n)$ space, and behaves as follows. If A and B are anagrams, then the algorithm always outputs YES, and if they are not, it outputs NO with probability at least $99/100$. You may assume that performing addition, multiplication, and mod operations on integers in the range 0 through $O((n + M)^2)$ takes unit time.

(20 points)

Problem 3. Alice and Bob are delivering for the Pizza store that has n orders today. From past experience, they both know that if Alice delivers the i -th order, the tip would be $A[i]$ dollars, and if Bob delivers, the tip would be $B[i]$ dollars. Alice and Bob are close friends, and their goal is to maximize the total tip they can get and then distribute it evenly between themselves. However, they do have a constraint: Alice can only handle at most a of the orders, and Bob can only handle at most b of the orders.

Design an $O(nab)$ time dynamic programming algorithm that given the arrays $A[1 : n]$, $B[1 : n]$, and integers $1 \leq a, b \leq n$, finds the largest amount of tips that Alice and Bob can collect together subject to the given constraints. Note that Alice and Bob can choose to skip deliveries.

Example: Suppose $A = [2, 4, 5, 1]$, $B = [1, 7, 2, 5]$, $a = 2$ and $b = 1$. Then the answer is 14 by Alice delivering 2, 3 and Bob delivering 4 (so $4 + 5 + 5 = 14$), or Alice delivering 1, 3 and Bob delivering 2 ($2 + 5 + 7 = 14$).

(20 points)

Problem 4. A farmer is analyzing his land to determine where to plant apple trees. Each acre of land can be considered as *suitable* or *unsuitable* to grow apples depending on the quality of soil, previous crop growth, and moisture levels. The farmer's goal is to maximize the largest area to grow apples. The constraint is that that chosen plot of land must be a square that consists entirely of suitable acres.

Specifically, you are given an array $A[1..m, 1..n]$, where each entry is either 0 or 1. A suitable acre at position $A[i, j]$ would have value 1; likewise, an unsuitable acre would have value 0. Design an $O(mn)$ dynamic programming algorithm that given the land of size $m \times n$ acres stored within the array A , returns the area of the largest possible square-shaped entirely suitable farmland.

Example: Suppose $A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$. The largest square consisting of entirely 1's is the

2×2 square at the top left corner, so the correct output is 4. (20 points)

Problem 5. An office has n employees, numbered 1 through n . Each employee has their own cubicle, and the cubicles are arranged in a straight line, numbered 1 through n such that employee i is in cubicle numbered i . At the end of the year, each employee i is given a performance rating p_i that is a positive integer. The manager of the office has been tasked with using the performance ratings to determine a performance bonus for each employee. Specifically, this task involves assigning each employee $i \in [n]$ an integer bonus point value b_i that is subject to the following constraints:

- Each worker receives at least one bonus point, that is, $\forall i \in [n], b_i \geq 1$.
- To minimize envy, it is required that if an employee has a strictly higher performance rating than an immediate neighbor, then they must get assigned strictly more bonus points than that immediate neighbor:

$$\forall i \in [n - 1], p_i < p_{i+1} \implies b_i < b_{i+1}$$

$$\forall i \in [n - 1], p_i > p_{i+1} \implies b_i > b_{i+1}$$

The manager wishes to compute a bonus point assignment that minimizes the sum $\sum_{i \in [n]} b_i$ subject to the constraints above. Design an $O(n)$ time algorithm to compute an optimal bonus point assignment. You can assume that the employee performance ratings are given to you as an array $P[1..n]$ such that $P[i] = p_i$ for $i \in [n]$. Your algorithm should output an array $B[1..n]$ such that $B[i] = b_i$ for $i \in [n]$ where $b_1, b_2, \dots, b_n$ represent an optimal assignment of bonus points. You may assume that all entries in the array $P[1..n]$ are *distinct* integers. (20 points)